

Software Design Document
AI Based University Queries Chatbot



Submitted by

Muhammad Hamad Bilal (S23BINFT1M01199)

Submitted to

DR. Asadullah Khan

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Summery

The Software Design Document (SDD) outlines the architecture, components, and functionality of the AI-Based University Queries Chatbot, developed using Python with Django, OpenAI's GPT API, and SQLite. The system is designed to assist students and visitors by answering university-related questions such as admission details, fee structures, contact information, and departmental data. The chatbot first checks for predefined answers stored in the database; if no match is found, it intelligently responds using OpenAI's GPT. The document covers all major aspects including system overview, architecture, use case, class and activity diagrams (in text format), data design (with tables for users, queries, and university info), and UI structure. It also maps functional requirements from the SRS to specific components in the system and concludes with tools used, assumptions, and future improvement ideas such as multilingual support, voice input, and mobile compatibility. The SDD serves as a complete technical blueprint for implementing and maintaining the chatbot.

Table of Content

1.Introduction	4
2.System Overview	4
3.System Architecture	5
3.1 Architectural Components	5
3.2 Workflow Description	5
4.System Design	6
4.1 Use Case Diagram	6
4.2 Class Diagram	7
4.3 Sequence Diagram	7
4.4 Activity Diagram	8
5.Data Design	8
5.1 Data Storage	8
5.2 Database Table	8
6.Component Design	9
6.1 Major Components	9
6.2 Component Interaction Summary	9
7.User Interface Design	10
8.Requirements Mapping	10
9.Appendices	11

1.Introduction

This Software Design Document (SDD) provides a complete technical blueprint for the *AI-Based University Queries Chatbot*, a web-based system developed for University . The chatbot is designed to assist students and visitors by providing instant, intelligent responses to university-related questions such as admissions, departments, fee structure, contact information, and academic schedules. The system is developed using Python with Django, integrating the OpenAI GPT API for natural language understanding and response generation, and utilizes SQLite as the backend database for storing structured university data and interaction logs. When a user submits a query, the system first checks for a matching response in its local database; if no match is found, it dynamically generates a human-like answer using the GPT API. This document defines the complete design architecture including component design, data models, interface layout, and textual diagrams such as use case, class, sequence, activity, and ER diagrams. The objective of this document is to ensure that all functional and non-functional requirements outlined in the Software Requirements Specification (SRS) are translated into a structured and maintainable software solution. Tools and technologies used in this project include Python 3.x, Django framework, SQLite database, OpenAI API, HTML/CSS, and GitHub. This document follows the IEEE standards for software design documentation and is intended for use by developers, testers, project supervisors, and future maintainers.

2.System Overview

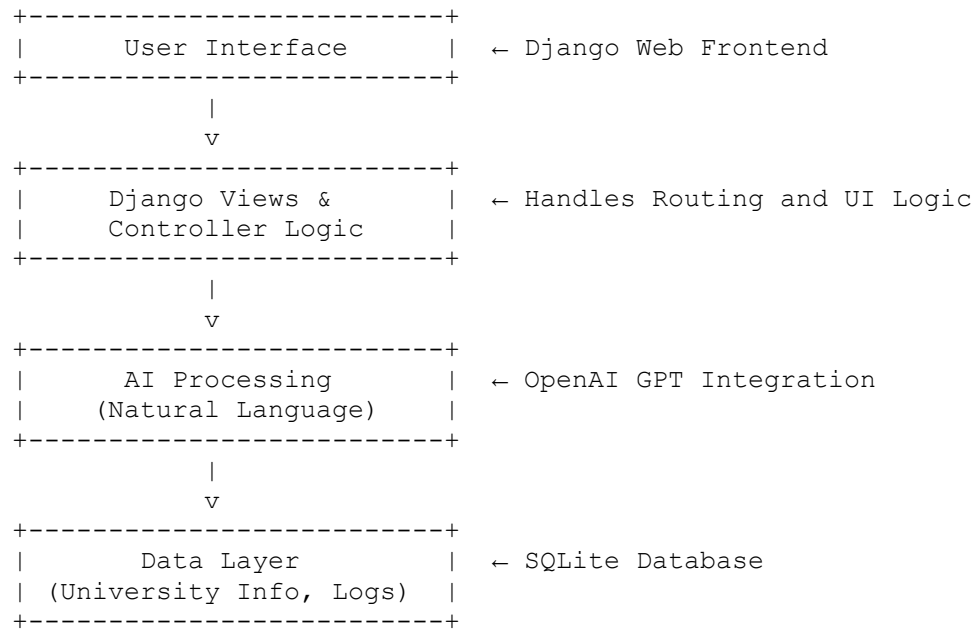
The *AI-Based University Queries Chatbot* is structured as follows:

- **Frontend & Backend:** A single cohesive framework built using **Django**. Django handles both user interface and server-side logic.
- **AI Layer:** OpenAI GPT API processes and generates intelligent responses to student queries.
- **Database:** SQLite stores predefined responses, departments, events, contact info, and logs of user interactions.
- **Functionality:**
 - Accepts user queries via a web interface
 - Matches queries against the database
 - If no match, passes the query to GPT for intelligent response
 - Displays results in real-time

3. System Architecture

The architecture of the *AI-Based University Queries Chatbot* follows a layered design:

3.1 Architectural Components



3.2 Workflow Description

1. **User** submits a query via the Django frontend.
2. **View Logic** routes the query to either:
 - A matching answer from the SQLite database (if available), or
 - The **OpenAI GPT API** for a generated response.
3. The **response** is returned to the UI and optionally saved to logs.

4. System Design

4.1 Use Case Diagram

□ **Primary Actor:** Student/User

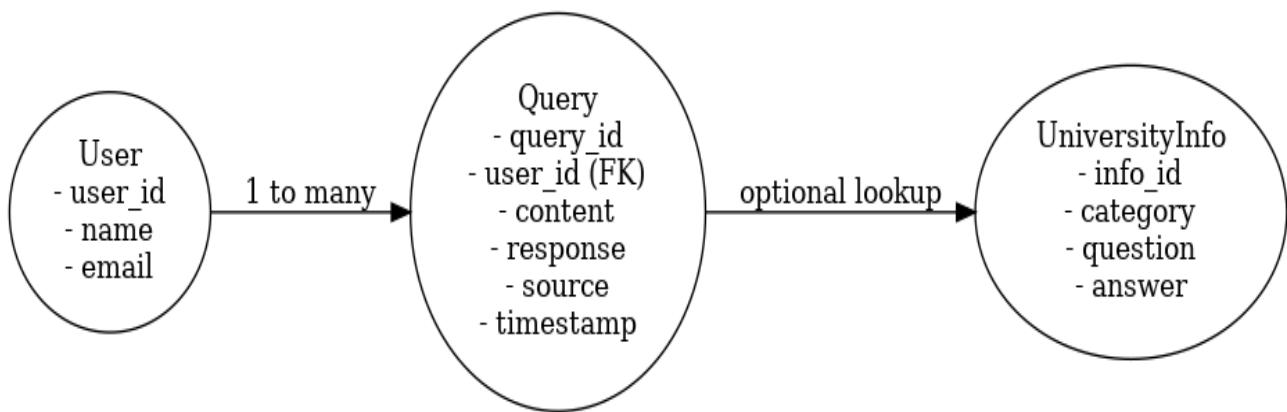
□ **Use Cases:**

- Submit Query
- View Response
- Browse Predefined FAQs
- Get Contact Information
- View Departments & Admissions Info

[Student]

|
|-----> (Submit Query)
|-----> (Browse FAQs)
|-----> (Get Department Info)
|-----> (View Contact Info)

4.2 Class Diagram



5. Data Design

5.1 Data Storage

The system uses **SQLite** to store:

- Predefined university-related information
- User queries and AI responses
- Basic user session logs

5.2 Database Tables

Field	Type	Description
user_id	INTEGER	Primary Key
name	TEXT	User's name
email	TEXT	Optional user email

6. Component Design

6.2 Major Components

Query Handle

- Accepts user input from frontend
- Determines whether to fetch from DB or OpenAI
- Logs response

AI Integration Module

- Sends queries to OpenAI's GPT API
- Formats and returns intelligent answers

Database Access Layer

- Interfaces with SQLite using Django ORM
- Fetches predefined university data
- Records new queries/responses

User Interface

- Built in Django templates + views
- Inputs, responses, and chatbot layout

6.2 Component Interaction Summary

```
User → Django View → Query Handler
Query Handler → SQLite DB (Check match)
                └─ or ─→ OpenAI API (If no match)
                        → Django Template (Response display)
```

7. User Interface Design

7.1 Main Features

- **Homepage:** Contains input box for entering university-related questions.
- **Chat Interface:** Mimics chatbot-style conversation layout.
- **Response Box:** Displays AI or database-based responses.
- **Help Section:** Suggests sample questions or categories.
- **About/Contact Page:** Contains IUB contact info or chatbot description.

7.2 Wireframe Overview (Textual)

| **AI-Based University Chatbot (Header)** |

|-----|

| **Ask your question below:** |

| [_____] [Submit] |

| _____ |

| **Chat:** |

| **User: "When is the last date of admission?"** |

| **Bot: "The last date is June 30, 2025."** |

| _____ |

| **Suggested: [Admission] [Departments]** |

8. Requirements Mapping

This section maps the SRS functional requirements to specific design components, ensuring traceability from requirements to implementation.

Requirement (from SRS)	Mapped Design Component
User submits query via interface	UI Design (Section 7), Django Views
Query is checked against stored data	Data Design → university_info Table
If not found, use AI to generate answer	AI Integration Module
Response is logged with source	Database Design → queries Table

User can access university information (fees, schedule)	UI + University Info Table (SQLite)
System provides intelligent, human-like responses	OpenAI GPT Integration
Secure, simple web-based platform	Django Framework (Built-in security)
Simple and intuitive user interface	UI Wireframe (Section 7.2)

9. Appendices

9.1 Tools & Technologies Used

Tool/Technology	Purpose
Python 3.x	Core programming language
Django	Web framework (Frontend + Backend)
SQLite	Lightweight relational database
OpenAI API	AI-powered question-answering engine
HTML/CSS	Frontend styling
GitHub	Version control and collaboration

9.2 Assumptions

- The user will have access to the internet.
- The OpenAI API key will be valid and active.
- Questions are in English and related to the university only.

9.3 Future Enhancements

- Add voice input support for queries
- Include multilingual support (e.g., Urdu)
- Add chatbot mobile version using React Native or Flutter
- Train GPT with university-specific datasets

